

Programování grafických aplikací základní operace s obrazem

Radek Richtr

Institute of Information Theory and Automation
Academy of Sciences of the Czech Republic
Prague, Czech Republic

August 14, 2026



Outline I

① Rovinná grafika

Obraz

2D objekty

② Transformace

Barvy

Geometrie

③ Konvoluce

Definice

Okrajový jev

Konvoluční filtry

④ Doplnění: diskrétní obraz a filtrace

Od spojité scény k pixelům

Konvoluce jako lokální lineární operace

⑤ Doporučené zdroje

Cíle přednášky

Po absolvování přednášky by měl student umět:

- rozlišit spojitou scénu, vzorkovaný obraz a kvantované hodnoty pixelů;
- vysvětlit rasterizaci základních 2D objektů a geometrickou spojitost křivek;
- zapsat diskrétní 2D konvoluci a zvolit vhodnou okrajovou podmínku;
- odlišit vyhlazení, detekci hran a ostření podle vlastností jádra.

Intro

- teoretická přednáška

Intro

- teoretická přednáška
- přehled algoritmických podkladů

Intro

- teoretická přednáška
- přehled algoritmických podkladů
- ... a matematických podkladů pro grafické algoritmy

Intro

- teoretická přednáška
- přehled algoritmických podkladů
- ... a matematických podkladů pro grafické algoritmy
- matematika je základ, bez kterého to nejde (alespoň její intuitivní pochopení)

Matematické minimum

- rovnice, parametrické rovnice

Matematické minimum

- rovnice, parametrické rovnice
- sumy, integrály

Matematické minimum

- rovnice, parametrické rovnice
- sumy, integrály
- matice, násobení matic

Matematické minimum

- rovnice, parametrické rovnice
- sumy, integrály
- matice, násobení matic
- transformace

Matematické minimum

- rovnice, parametrické rovnice
- sumy, integrály
- matice, násobení matic
- transformace
- determinant

Reprezentace obrazu

- Formáty
 - bmp
 - gif
 - jpg
 - png
 - atd.

Reprezentace obrazu

- A co video?
- Sekvece framů?
- Jaký datový typ a rozsah, organizace v paměti?
- Programová reprezentace?

2D objekty

- Úsečky
- Kružnice a elipsa (Bresenham)
- Vyplňování (semínkové vyplňování)
- Polygon
- 4-okolí a 8-okolí
- Ořezávání (Cohen-Sutherland)

2D objekty

- Objekty mohou být vektorové...

2D objekty

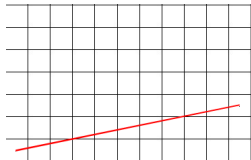
- Objekty mohou být vektorové...
- ale jsou vykreslené v rastru!

2D objekty

- Objekty mohou být vektorové...
- ale jsou vykreslené v rastru!
- Které z pixelů 'rosvítit'?

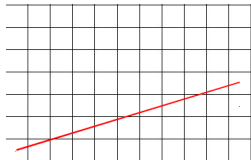
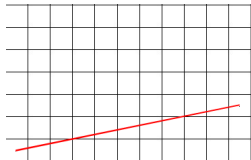
2D objekty

- Které pixely v rastru 'rozsvítit'?



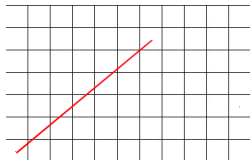
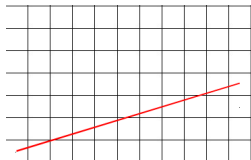
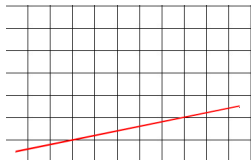
2D objekty

- Které pixely v rastru 'rozsvítit'?



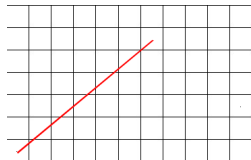
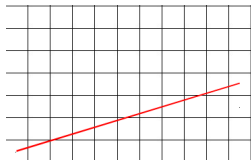
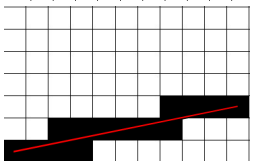
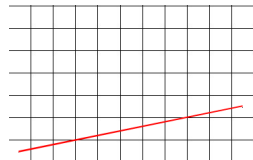
2D objekty

- Které pixely v rastru 'rozsvítit'?



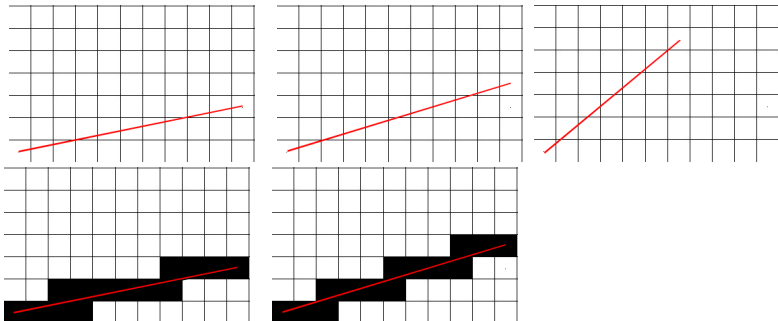
2D objekty

- Které pixely v rastru 'rozsvítit'?



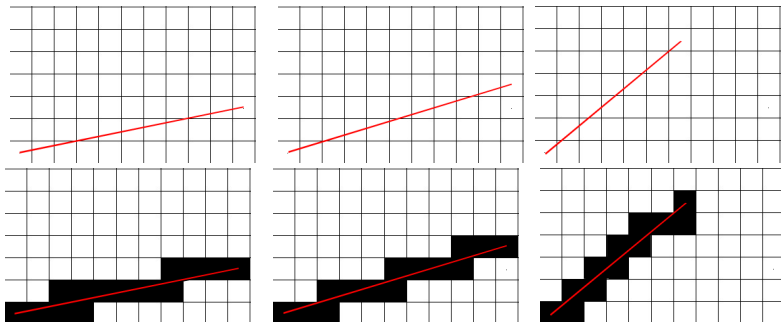
2D objekty

- Které pixely v rastru 'rozsvítit'?



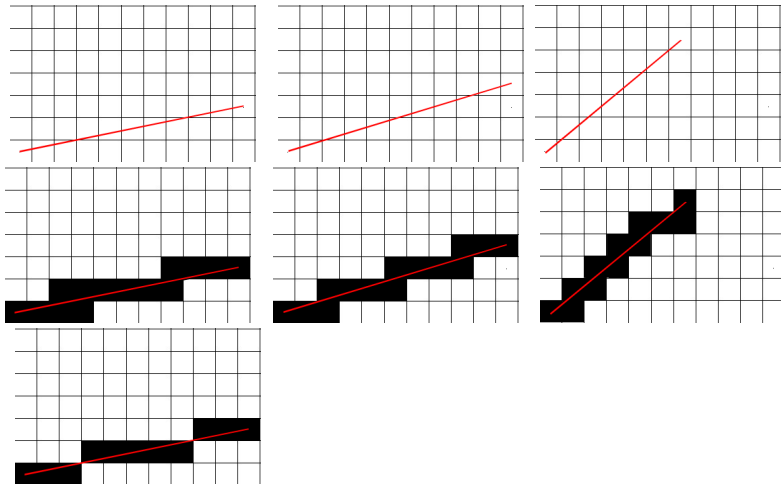
2D objekty

- Které pixely v rastru 'rozsvítit'?



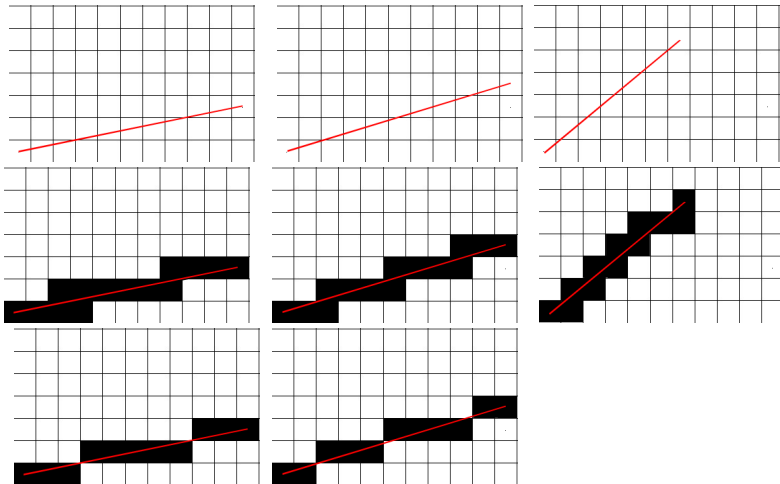
2D objekty

- Které pixely v rastru 'rozsvítit'?



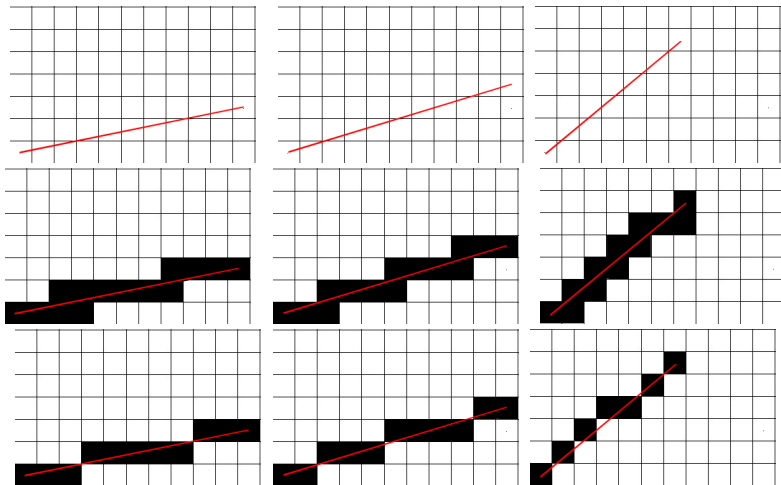
2D objekty

- Které pixely v rastru 'rozsvítit'?



2D objekty

- Které pixely v rastru 'rozsvítit'?

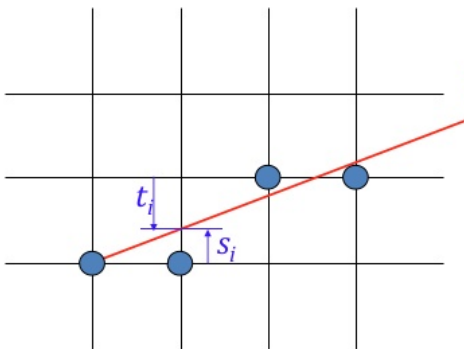


Bresenhamův algoritmus

- To, který pixel rastru vykreslit je potřeba vyřešit efektivně!
- A efektivně nejen na úrovni logiky:
 - dělení je náročná operace
 - a co teprve dělení neceločíselné
 - a co přesnost?
- Rychlý algoritmus, který je navíc celočíselný!

Bresenhamův algoritmus

- Nachází body nejbíže kreslené přímce a to pomocí *celočíslné aritmetiky*
- Funguje symetricky k určité řídící ose
- Rozhoduje, jestli je další pixel na stejné, nebo vyšší úrovni

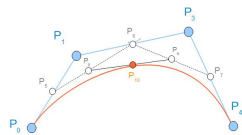
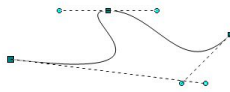
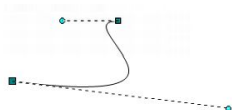


Bresenhamův algoritmus

```
var d = 2*dy - dx
var dy = y2-y1; var dx = x2-x1
var x = x1 ; var y = y1
while (x <= x2)
{
  Draw pixel at (x,y)
  x++
  if( d<0 )
    d += dy + dy
  else
  {
    d += 2*(dy-dx)
    y++
  }
}
```

2D objekty

- Kružnice a elipsy
 - obdobný problém jako u přímk
 - výhody ve větší symetrii
- Křivky
 - i křivky mohou být 3D!
 - rozdíl mezi 'křivkou' a 'náhodnou čarou'
- Polynomiální křivky
- Spline křivky
- Beziérové křivky
- NURBS



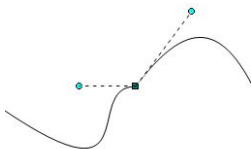
- Spojitost
 - očekávaný tvar 'budoucí' křivky na základě 'minulé'
 - stejný tečný vektor i jeho velikost
 - C^m parametrická spojitost stupně m
 - G^m geometrická spojitost stupně m

Spojitosť

- C^0 : Spojité napojení. Koncový bod první křivky a počáteční bod druhé křivky jsou totožné.
Animace pohybu: animovaný objekt nezmění skokem *polohu*.
- C^1 : Tečné vektory (první derivace) jsou si v bodě spojení rovny.
Animace pohybu: animovaný objekt nezmění skokem *směr*.
- C^2 : Druhé derivace v bodě spojení jsou si rovny.
Animace pohybu: animovaný objekt nezmění skokem *rychlost*.

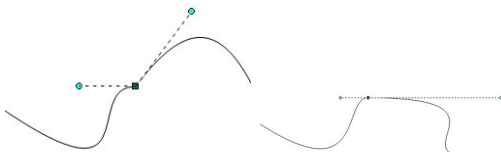
Spojitosť

- C^0 : Spojité napojení. Koncový bod první křivky a počáteční bod druhé křivky jsou totožné.
Animace pohybu: animovaný objekt nezmění skokem *polohu*.
- C^1 : Tečné vektory (první derivace) jsou si v bodě spojení rovny.
Animace pohybu: animovaný objekt nezmění skokem *směr*.
- C^2 : Druhé derivace v bodě spojení jsou si rovny.
Animace pohybu: animovaný objekt nezmění skokem *rychlost*.



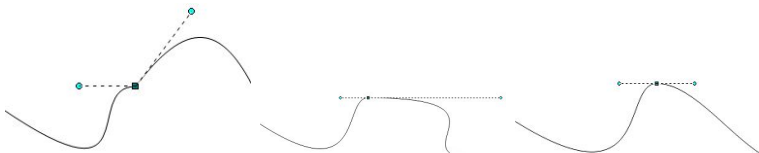
Spojitosť

- C^0 : Spojité napojení. Koncový bod první křivky a počáteční bod druhé křivky jsou totožné.
Animace pohybu: animovaný objekt nezmění skokem *polohu*.
- C^1 : Tečné vektory (první derivace) jsou si v bodě spojení rovny.
Animace pohybu: animovaný objekt nezmění skokem *směr*.
- C^2 : Druhé derivace v bodě spojení jsou si rovny.
Animace pohybu: animovaný objekt nezmění skokem *rychlost*.



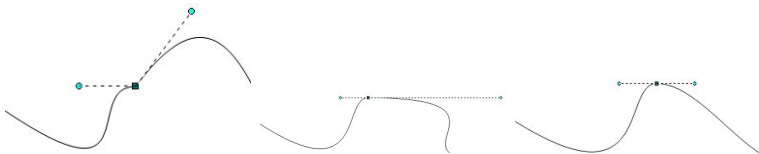
Spojitosť

- C^0 : Spojité napojení. Koncový bod první křivky a počáteční bod druhé křivky jsou totožné.
Animace pohybu: animovaný objekt nezmění skokem *polohu*.
- C^1 : Tečné vektory (první derivace) jsou si v bodě spojení rovny.
Animace pohybu: animovaný objekt nezmění skokem *směr*.
- C^2 : Druhé derivace v bodě spojení jsou si rovny.
Animace pohybu: animovaný objekt nezmění skokem *rychlost*.



Spojitosť

- C^0 : Spojité napojení. Koncový bod první křivky a počáteční bod druhé křivky jsou totožné.
Animace pohybu: animovaný objekt nezmění skokem *polohu*.
- C^1 : Tečné vektory (první derivace) jsou si v bodě spojení rovny.
Animace pohybu: animovaný objekt nezmění skokem *směr*.
- C^2 : Druhé derivace v bodě spojení jsou si rovny.
Animace pohybu: animovaný objekt nezmění skokem *rychlost*.



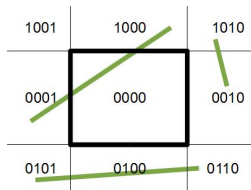
- G^0 - počáteční a koncový bod je totožný
- G^1 - tečné vektory jsou lineárně závislé

2D objekty

- Vyplňování hranice
 - řádkové se seznamem hran
 - semínkový algoritmus
- Pozor na spojitost v rastru
 - 4-okolí (návaznost při posunu v ose x nebo y)
 - 8-okolí (návaznost i diagonálně)
- Hranice $\times$ vnitřek
 - Obojí 4-okolí?
 - Obojí 8-okolí?
 - Hranice 8-okolí a vnitřek 4-okolí?
 - Hranice 4-okolí a vnitřek 8-okolí?

2D objekty

- Ořezávání useček
 - Cohen-Sutherland
- Koncové body p_1 a p_0 mají bitový kód - p_1 or $p_0 = 0 \Rightarrow$ úsečka v okně
 - p_1 and $p_0 \neq 0 \Rightarrow$ úsečka celá mimo
 - jinak $\Rightarrow$ úsečka prochází několika oblastmi
- jinak: Zkrácení úsečky
 - podle bitového kódu zvolíme další přímku, spočteme průsečík
 - opakovat dokud úsečka není celá v okně



Transformace barev

- Konstantní prah
 - pevný převod do (obvykle značně) omezené palety barev
 - paleta buď rovnoměrně rozdělená, nebo optimalizovaná
 - často například u tisku posterů, plakátů



Transformace barev

- Konstantní prah
 - pevný převod do (obvykle značně) omezené palety barev
 - paleta buď rovnoměrně rozdělená, nebo optimalizovaná
 - často například u tisku posterů, plakátů



Transformace barev

- Náhodné rozptýlení
 - intenzita zdrojové černé $\Rightarrow$ pravděpodobnost



Transformace barev

- Náhodné rozptýlení
 - intenzita zdrojové černé $\Rightarrow$ pravděpodobnost



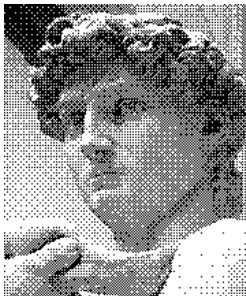
Transformace barev

- Maticové rozptýlení
 - pixel nestojí sám o sobě, převádí se malé oblasti
 - konkrétních rozptylovacích matic je mnoho



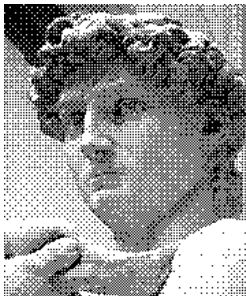
Transformace barev

- Maticové rozptýlení
 - pixel nestojí sám o sobě, převádí se malé oblasti
 - konkrétních rozptylovacích matic je mnoho



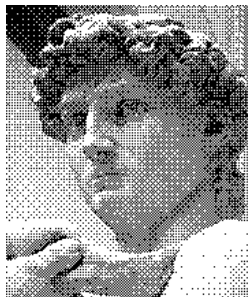
Transformace barev

- Maticové rozptýlení
 - pixel nestojí sám o sobě, převádí se malé oblasti
 - konkrétních rozptylovacích matic je mnoho



Transformace barev

- Maticové rozptýlení
 - pixel nestojí sám o sobě, převádí se malé oblasti
 - konkrétních rozptylovacích matic je mnoho



$$C_{in} = \{1, 2, 3, 4, 5\} \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 3 \\ 2 & 1 \end{bmatrix}$$

A jak by fungoval převod do ASCII?

Transformace barev

- Maticové rozptýlení
 - pixel nestojí sám o sobě, převádí se malé oblasti
 - konkrétních rozptylovacích matic je mnoho

$$M_{dispej} = \begin{bmatrix} 0 & 12 & 3 & 15 \\ 8 & 4 & 11 & 7 \\ 2 & 14 & 1 & 13 \\ 10 & 6 & 9 & 5 \end{bmatrix}$$

$$M_{tisk} = \begin{bmatrix} 1 & 5 & 9 & 2 \\ 8 & 12 & 13 & 6 \\ 4 & 15 & 14 & 10 \\ 0 & 11 & 7 & 3 \end{bmatrix}$$

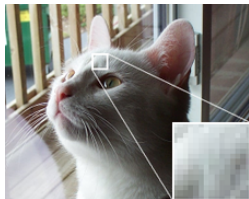
Transformace barev

- Distribuce zaokrouhlovací chyby
 - pixel nestojí sám o sobě
 - při převodu do omezené palety vznikne chyba
 - ta se *roz distributes* do ještě nepřevedených pixelů v okolí
- Floyd–Steinberg dithering - distribuce chyby + optimalizované paleta

$$\begin{bmatrix} & & & & \\ & & \star & \frac{7}{16} & \dots \\ \dots & \frac{3}{16} & \frac{5}{16} & \frac{1}{16} & \dots \end{bmatrix}$$

Transformace barev

Originál

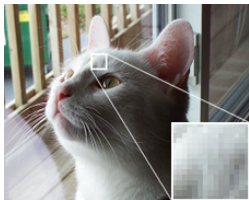


8bit prah.

8bit distr.

Transformace barev

Originál



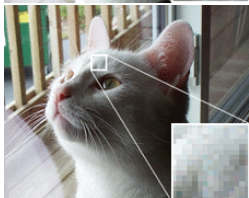
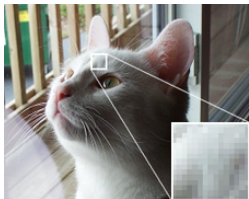
8bit prah.



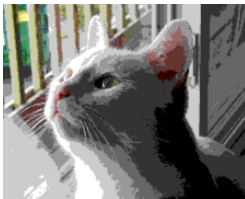
8bit distr.

Transformace barev

Originál



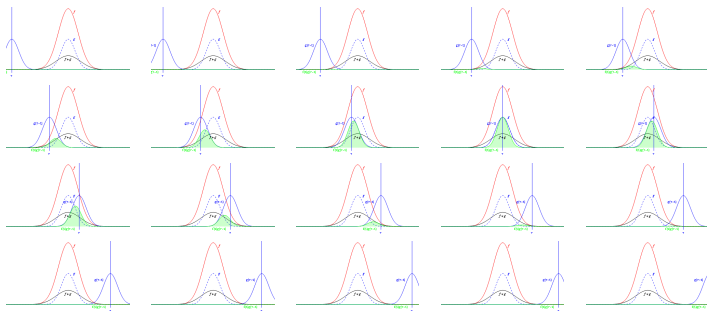
8bit prah.



8bit distr.



Spojitá konvoluce



Konvoluce

Základní vlastnosti

$$f * g = g * f \quad (3)$$

$$f * (g * h) = (f * g) * h \quad (4)$$

$$a(f * g) = (af) * g = f * (ag) \quad (5)$$

$$f * (g + h) = (f * g) + (f * h) \quad (6)$$

$$f * \delta = \delta * f = f \quad (7)$$

Diskrétní Konvoluce

- Diskrétní konvoluce (rastr)
- 1D

$$(f * g) = \sum_{m=-\infty}^{\infty} f[m] \cdot g[n - m] \quad (11)$$

$$f \quad \boxed{1 \quad 4 \quad 2 \quad 5} \quad g \quad \boxed{3 \quad 4 \quad 1} \quad c = f * g$$

$$\begin{array}{cccc} & & \boxed{1} & \boxed{4} & \boxed{2} & \boxed{5} \\ \boxed{1} & \boxed{4} & \boxed{3} & & & \end{array}$$

$$c[0] = 1 * 3 = 3$$

$$\begin{array}{cccc} & & \boxed{1} & \boxed{4} & \boxed{2} & \boxed{5} \\ & \boxed{1} & \boxed{4} & \boxed{3} & & \end{array}$$

$$c[1] = 1 * 4 + 4 * 3 = 16$$

$$\begin{array}{cccc} \boxed{1} & \boxed{4} & \boxed{2} & \boxed{5} \\ \boxed{1} & \boxed{4} & \boxed{3} & \end{array}$$

$$c[2] = 1 * 1 + 4 * 4 + 2 * 3 = 23$$

$$\begin{array}{cccc} \boxed{1} & \boxed{4} & \boxed{2} & \boxed{5} \\ & \boxed{1} & \boxed{4} & \boxed{3} \end{array}$$

$$c[3] = 4 * 1 + 2 * 4 + 5 * 3 = 27$$

$$\begin{array}{cccc} \boxed{1} & \boxed{4} & \boxed{2} & \boxed{5} \\ & & \boxed{1} & \boxed{4} & \boxed{3} \end{array}$$

$$c[4] = 2 * 1 + 5 * 4 = 22$$

$$\begin{array}{cccc} \boxed{1} & \boxed{4} & \boxed{2} & \boxed{5} \\ & & & \boxed{1} & \boxed{4} & \boxed{3} \end{array}$$

$$c[5] = 5 * 1 = 5$$

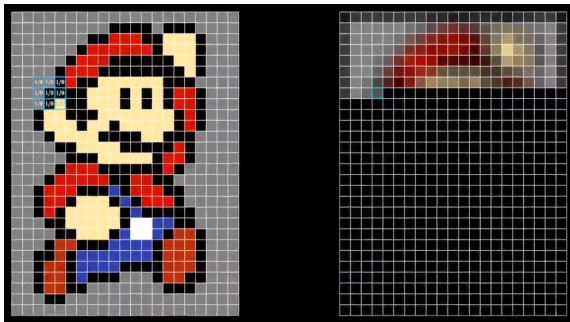
Diskrétní konvoluce

- Diskrétní konvoluce (rastr)
- 2D (obrázky)

$$(f * g) = \sum_{m=-\infty}^{\infty} f[m] \cdot g[n - m] \quad (12)$$



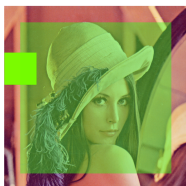
Diskrétní konvoluce



video

Okrajový jev

- Diskrétní konvoluce (rastr)
- 2D (obrázky)
- Okrajový jev: A co na kraji obrazu?
 - type valid
 - type same
 - type full



Okrajový jev

- Diskrétní konvoluce (rastr)
- 2D (obrázky)
- Okrajový jev: A co na kraji obrazu?
 - oblepení nulami (zero padding)
 - zrcadlové prodloužení (mirror extension)
 - periodické prodloužení (periodic/toroid extension)



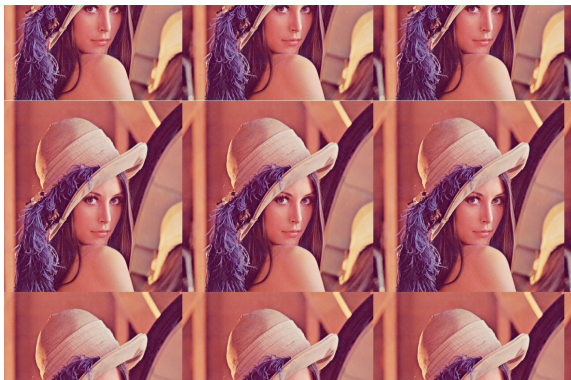
Okrajový jev

- Diskrétní konvoluce (rastr)
- 2D (obrázky)
- Okrajový jev: A co na kraji obrazu?
 - oblepení nulami (zero padding)
 - zrcadlové prodloužení (mirror extension)
 - periodické prodloužení (periodic/toroid extension)



Okrajový jev

- Diskrétní konvoluce (rastr)
- 2D (obrázky)
- Okrajový jev: A co na kraji obrazu?
 - oblepení nulami (zero padding)
 - zrcadlové prodloužení (mirror extension)
 - periodické prodloužení (periodic/toroid extension)



Konvoluční filtry

- Prosté a vážené průměrování
- Odšumění
- Detekce hran
- Ostření
- Filtry ve frekvenční oblasti (příště)

Odšumění

- Prosté a vážené průměrování
- Odšumění
- Součet roven 1
 - proč?

$$\frac{1}{9} \times \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Odšumění

- Prosté a vážené růměrování
- Gaussův šum
- Součet roven 1
- proč?

$$\frac{1}{16} \times \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Detekce hran

- Sobelův operátor
 - první derivace
 - směrově orientovaný
 - dvojice masek h a $\bar{h}$
- Součet roven 0
 - Proč?

$$h = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad \bar{h} = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

$$|G| = \sqrt{h^2 + \bar{h}^2}$$

Detekce hran

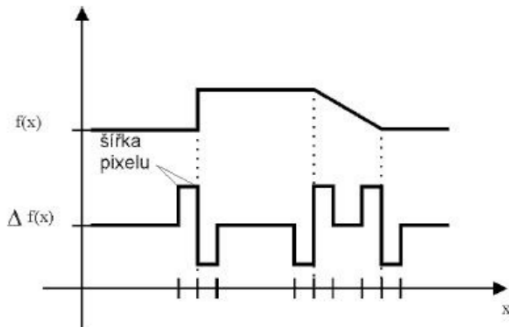
- Sobelův operátor
 - první derivace
 - směrově orientovaný
 - dvojice masek h a $\bar{h}$

$$h = \begin{bmatrix} -2 & -1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & 2 \end{bmatrix} \quad \bar{h} = \begin{bmatrix} 0 & -1 & -2 \\ 1 & 0 & -1 \\ 2 & 1 & 0 \end{bmatrix}$$

$$|G| = \sqrt{h^2 + \bar{h}^2}$$

Detekce hran

- Laplaceův operátor
 - druhá derivace
 - dvojí reakce na hranu



Co se děje na hranici obrazu?

Pro výpočet poblíž okraje chybějí vzorky. Volba rozšíření je součástí algoritmu:

- **nuly**: jednoduché, ale vytvářejí tmavý lem;
- **replikace**: prodlužuje krajní hodnotu;
- **zrcadlení**: obvykle omezuje skok na hranici;
- **periodické rozšíření**: přirozené pro DFT, nevhodné pro nenavazující obraz.

Výstup *valid* používá jen úplná okolí; *same* zachovává rozměr pomocí zvolené okrajové podmínky.

Separabilní filtr a výpočetní cena

Pokud lze jádro zapsat jako $h[i, j] = u[i]v[j]$, pak

$$I * h = (I * u) * v.$$

- Přímá konvoluce s jádrem $k \times k$ stojí přibližně k^2 operací na pixel.
- Dvě jednorozměrné konvoluce potřebují přibližně $2k$ operací na pixel.
- Gaussovské vyhlazení je separabilní; velká jádra jsou proto výrazně levnější.

Od derivace k hraně a ostření

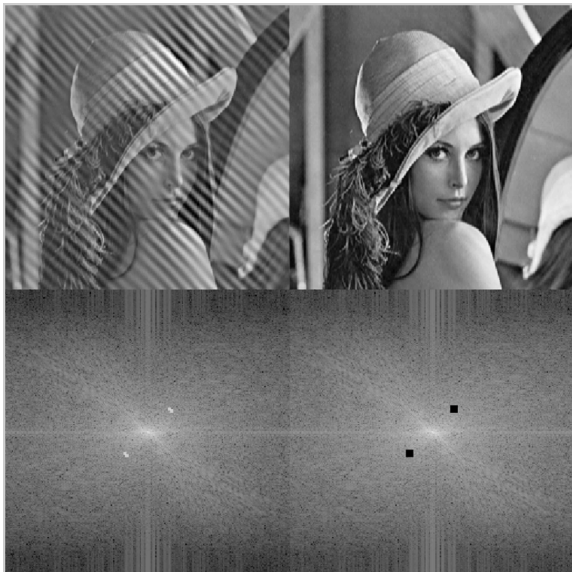
- První derivace (Sobel, Scharr) odhaduje gradient $\nabla I = (I_x, I_y)$.
- Velikost $\|\nabla I\|_2 = \sqrt{I_x^2 + I_y^2}$ vyjadřuje sílu hrany, $\text{atan2}(I_y, I_x)$ její orientaci.
- Laplaceův operátor $\nabla^2 I = I_{xx} + I_{yy}$ je izotropní, ale citlivý na šum.
- Unsharp masking ostří obraz jako $I_{\text{sharp}} = I + \alpha(I - G_\sigma * I)$.

Derivaci proto obvykle předchází vyhlazení; jinak zesílíme i vysokofrekvenční šum.

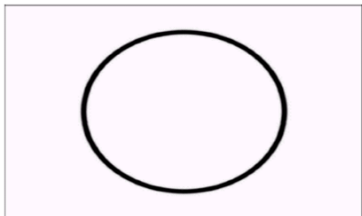
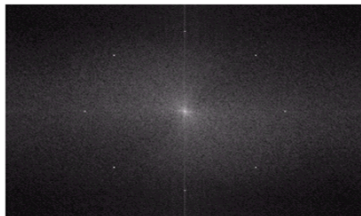
Doporučené zdroje I

- [1] Rafael C. Gonzalez and Richard E. Woods.
Digital Image Processing.
Pearson, 4 edition, 2018.
Základní učebnice zpracování obrazu, konvoluce a frekvenční filtrace.
- [2] Alan V. Oppenheim and Ronald W. Schafer.
Discrete-Time Signal Processing.
Prentice Hall, 3 edition, 2010.
Podrobný výklad diskrétních systémů a Fourierovy transformace.

Příště



Příště



Zdroje

...

